触摸拍照

触摸拍照

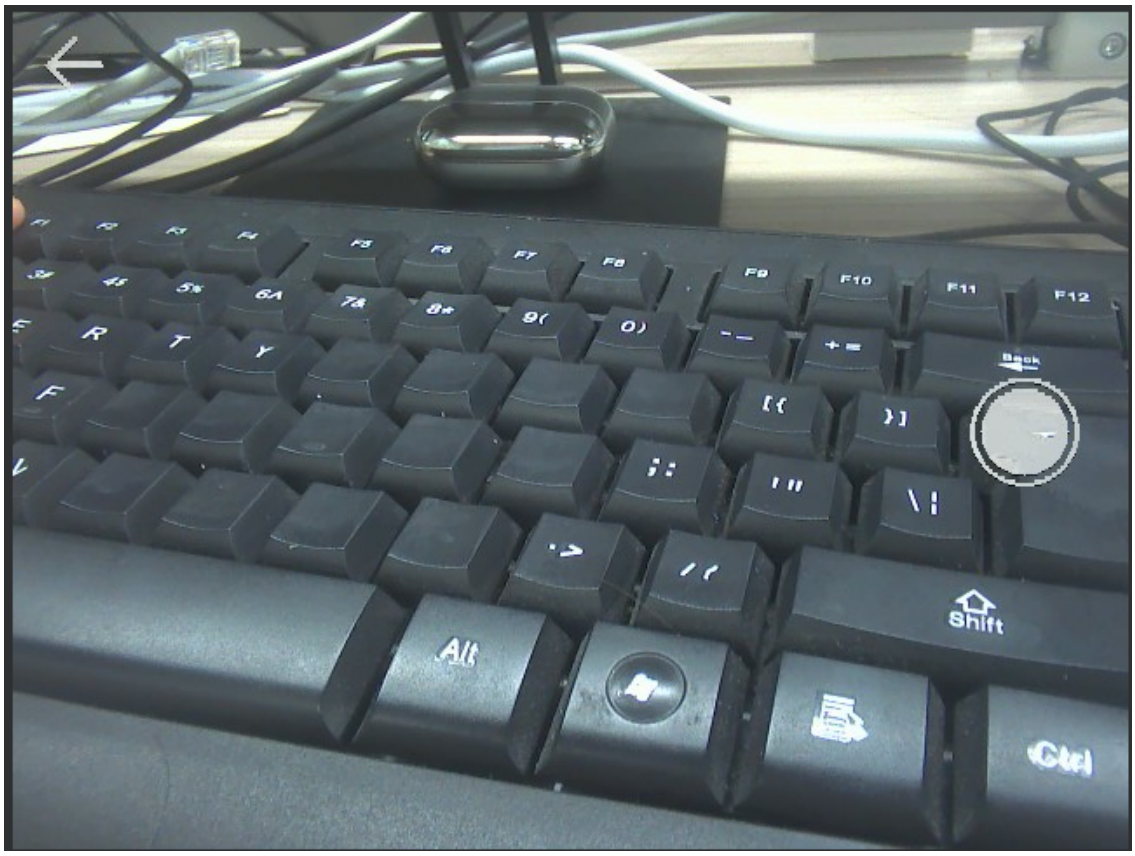
- 运行效果
- 代码讲解
  - 1. 导入必要的模块
  - 2. 初始化触摸传感器
  - 3. 辅助函数：确保目录存在
  - 4. 定义 `App` 类
    - 4.1 初始化界面和资源
    - 4.2 创建用户界面
      - 4.2.1 左侧面板：文本和按钮
      - 4.2.2 右侧面板：预览图像
  - 5. 相机预览与拍照功能
    - 5.1 打开相机预览
    - 5.2 相机预览主循环
    - 5.3 拍照并保存
  - 6. 退出相机模式

运行效果

- 注：本例程未提供离线运行版本，需要从出厂自带的GUI程序中启动

  本节例程代码位于[\sdcard\apps\camera]

  运行效果如图所示



  触摸屏幕右侧的白色按钮，即可拍照并保存到data目录下

代码讲解

1. 导入必要的模块

```
import lvgl as lv
import os, gc
import time
import binascii
from ybMain.base_app import BaseApp
from media.media import *
from media.display import *
from machine import TOUCH
from ybUtils.Configuration import Configuration
```

- `lvgl`：用于创建图形用户界面的轻量级GUI库。
- `os`，`gc`：用于文件系统操作和垃圾回收。
- `time`：用于时间控制。
- `binascii`：用于生成随机文件名。
- `ybMain.base_app.BaseApp`：基础应用程序类，继承自此类的应用程序可以集成到应用管理器中。
- `media.media` 和 `media.display`：用于屏幕显示和媒体资源管理。
- `machine.TOUCH`：用于初始化和操作触摸传感器。
- `ybUtils.Configuration`：用于加载配置文件。

2. 初始化触摸传感器

```
tp = TOUCH(0)
```

- `TOUCH(0)`：在引脚 0 上初始化触摸传感器，`tp` 用于后续读取触摸数据。

3. 辅助函数：确保目录存在

```
def ensure_dir(directory):
    if not directory or directory == '/':
        return
    directory = directory.rstrip('/')
    try:
        os.stat(directory)
        return
    except OSError:
        print(f"Directory does not exist, creating: {directory}")
        if '/' in directory:
            parent = directory[:directory.rindex('/')]
            if parent and parent != directory:
                ensure_dir(parent)
        try:
            os.mkdir(directory)
        except OSError as e:
            print(f"Error creating directory {directory}: {e}")
            try:
                os.stat(directory)
            except:
                print(f"Still does not exist: {directory}")
    except Exception as e:
        print(f"Unexpected error: {e}")
```

- `ensure_dir()`：递归检查并创建目录，确保指定的路径存在。如果目录不存在，会尝试创建它。

4. 定义 `App` 类

```
class App(BaseApp):
    def __init__(self, app_manager):
```

- `App` 类继承自 `BaseApp`，是相机应用程序的核心，负责界面初始化、触摸事件处理和相机操作。
- 构造函数中加载背景图标图片（如果存在），并初始化配置和文本内容。

4.1 初始化界面和资源

```
def initialize(self):
    self.save_path = "/data/photo/"
    self.prefix = binascii.hexlify(os.urandom(5)).decode()
    self.i = 1
```

- `save_path`：设置照片保存路径为 `/data/photo/`。
- `prefix`：生成一个随机的5字节十六进制字符串作为文件夹前缀。
- `i`：照片文件编号，从1开始递增。

4.2 创建用户界面

```
content = lv.obj(self.screen)
content.set_size(lv.pct(100), 420)
content.align(lv.ALIGN.BOTTOM_MID, 0, 0)
```

- 创建主容器 `content`，设置为屏幕底部中间对齐，宽度100%，高度420像素。
- 设置为水平布局（`FLEX_FLOW.ROW`），包含左侧面板和右侧面板。

4.2.1 左侧面板：文本和按钮

```
left_panel = lv.obj(content)
left_panel.set_size(lv.pct(35), lv.pct(100))
left_panel.set_flex_flow(lv.FLEX_FLOW.COLUMN)
```

- 创建左侧面板 `left_panel`，占主容器宽度的35%。
- 添加介绍文本标签和保存路径标签，文本内容从配置文件中获取。
- 添加“打开相机”按钮，点击后触发 `open_camera()` 方法。

4.2.2 右侧面板：预览图像

```
right_panel = lv.obj(content)
right_panel.set_size(lv.pct(65), lv.pct(100))
img = lv.img(right_panel)
```

- 创建右侧面板 `right_panel`，占主容器宽度的65%。
- 加载并显示相机预览图（如果存在），设置为居中对齐并保持原始比例。

5. 相机预览与拍照功能

5.1 打开相机预览

```
def open_camera(self, event):
    gc.collect()
    img2 = image.Image(640, 480, image.RGB565)
    img2.clear()
```

- 调用垃圾回收器释放内存。
- 创建一个分辨率为640x480的图像 `img2`，并清除内容。
- 在图像上绘制拍摄按钮（右侧中间的圆圈）和返回按钮（左上角的箭头）。

```
Display.show_image(img2, 0, 0, Display.LAYER_OSD0)
```

- 将绘制好的界面显示在屏幕上（`LAYER_OSD0` 图层）。

5.2 相机预览主循环

```
while True:
    point = tp.read(1)
    if len(point):
        pt = point[0]
        if pt.event == TOUCH.EVENT_DOWN:
            if pt.x > 520 and pt.y > 150 and pt.x < 640 and pt.y < 300:
                self.take_picture(img)
                img.draw_rectangle(4, 4, 640 - 8, 480 - 8, (200, 0, 0),
thickness=8)
                Display.show_image(img, 0, 0, Display.LAYER_OSD3)
                time.sleep_ms(100)
            elif pt.x < 60 and pt.y < 60:
                self.exit_demo()
                time.sleep_ms(10)
                break

    img = self.pl.sensor.snapshot(chn=CAM_CHN_ID_1)
    Display.show_image(img, 0, 0, Display.LAYER_OSD3)
    time.sleep_ms(10)
```

- 持续读取触摸数据：
  - 如果触摸点在右侧拍摄按钮区域（`x > 520, y > 150, x < 640, y < 300`），调用 `take_picture()` 拍照，并显示一个红色边框作为反馈。
  - 如果触摸点在左上角返回区域（`x < 60, y < 60`），调用 `exit_demo()` 退出相机模式。
- 使用 `snapshot()` 获取相机预览图像，并实时显示在屏幕上（`LAYER_OSD3` 图层）。
- 每10毫秒更新一次。

5.3 拍照并保存

```
def take_picture(self, img):
    ensure_dir(self.save_path + str(self.prefix) + "/")
    path = self.save_path + str(self.prefix) + "/" + str(self.i) + ".jpg"
    self.i += 1
    img.save(path)
    time.sleep_ms(2)
```

- 调用 `ensure_dir()` 确保保存路径存在。
- 生成保存路径和文件名（格式为 `{save_path}/{prefix}/{i}.jpg`），并递增文件编号。
- 将当前预览图像保存为JPEG文件。

6. 退出相机模式

```
def deinitialize(self):
    img2 = image.Image(640, 480, image.RGB565)
    img2.clear()
    Display.show_image(img2, 0, 0, Display.LAYER_OSD3)

def exit_demo(self):
    self.deinitialize()
    return
```

- `deinitialize()`：创建一个空白图像并显示在 `LAYER_OSD3` 图层，清除相机预览。
- `exit_demo()`：调用 `deinitialize()` 清理资源并退出相机模式。